

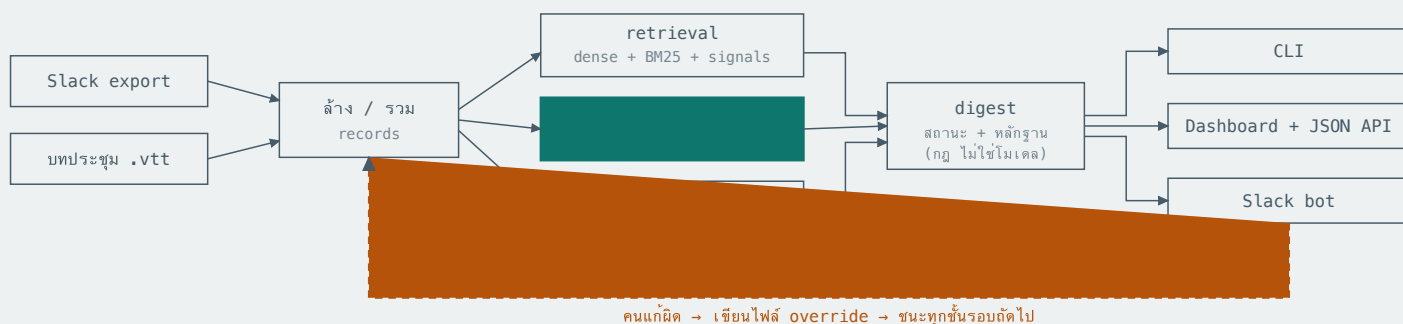
ตาม — รู้ว่าจะไรติด ก่อนเขา standup

อ่าน Slack, บันทึกประชุม และทิกเก็ต YouTrack รวมเป็น “งาน” หนึ่งชิ้น แล้วบอกว่าอะไรติด ติดเพราะอะไร และ **หลักฐานอยู่ข้อความไหน** — สถานะคำนวณด้วยกฎที่ตรวจสอบได้ ไม่ได้ให้โมเดลภาษาเดา

01

ทำงานยังไง

เส้นทางเดียวจากข้อความดิบไปถึงสามหน้าจอ ทุกกล่องเป็นโมดูลที่รันแยกได้และตรวจสอบผลตัวเอง



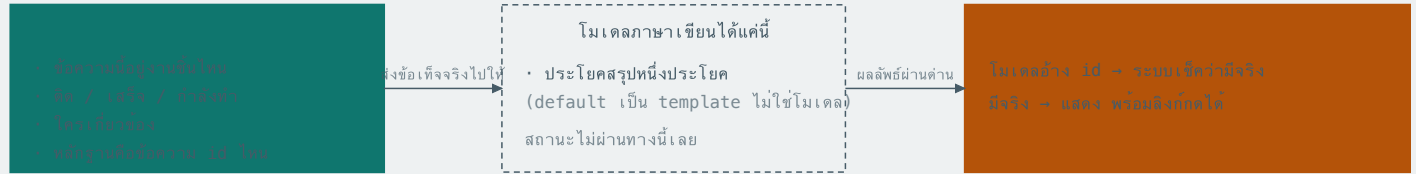
ทุกกล่องรันแยกได้:

```
python3 -m tam.<area>.<module> --help · 21 CLI ตอบ --help ทั้งหมด
```

รูป 1 — สองแหล่งเข้า สามสายวิเคราะห์ หนึ่งจัดรวม สามหน้าจอออก เส้นประคือวงจรที่ทำให้ต่างจากถาม AI แล้วจบ: สิ่งที่เคยย้อนกลับเข้าไปเป็นชั้นที่ชนะทุกชั้น · **แหล่งที่สามเข้ามาแล้ว** — ทิกเก็ต YouTrack ถูกแปลงเป็น record แยกเดียวกันที่กล่อง “ล้าง / รวม” จึงไม่มี pipeline ที่สอง (ผังสามแหล่งเดิม ๆ อยู่ใน [architecture.html](#))

กลไกที่เป็นหัวใจ

แยกให้ชัดว่าจะไรคำนวณ อะไรเขียน โมเดลภาษาได้เขียนแค่ประโยคเดียว และประโยคนั้นยังต้องผ่านการตรวจ



ผลที่ตามมา — ระบบกลับตอบว่า “ไม่มี”

เคยรายงาน 0 blocked บน 936 ข้อความ และรายงาน 0 ตรง ๆ ไม่แต่งขึ้นมา

วันนี้อ่าน cue ระดับบรรทัดแล้ว จึงได้ blocked 7 จาก 38 งาน — ทุกอันมีบรรทัดที่พิสูจน์

รูป 2 — เส้นแบ่งความรับผิดชอบ ทุกอย่างที่เป็นข้อเท็จจริงคำนวณจากกฎ โมเดลได้เขียนแค่ประโยค และประโยคนั้นต้องอ้างข้อความที่มีอยู่จริงถึงจะขึ้นหน้าจอ

ผลลัพธ์บนข้อมูลจริง

1,301

record ที่ระบบใช้จริง

จากไฟล์ 1,492 record — Slack 1,079 · reply ในเรสด 191 (ตัดออกตาม default) · ทึกเกิด YouTrack 196 · ประชุม 24

38

งานในหน้าต่าง 7 วัน

22 กำลังทำ · **7 ติดอยู่** · 9 ปิดแล้ว
· ทั้งคลังทั้งหมด 66 งาน

13

งานที่ได้ ticket จริงเป็นชื่อ

linker เจอ key ในข้อความเอง เช่น PROJ-142 · ที่เหลือใช้ hash ที่คงที่ · เทียบกับ YouTrack ได้ทั้ง 13 งาน

198

เทสอัตโนมัติ

Python 124 · TypeScript 74 · พิสูจน์แล้วว่าจับ regression ได้จริง

วัดจาก

curl localhost:8899/api/health → 1301 records · 38 topics · BAAI/bge-m3

cd pipeline && python3 -m pytest -q → 124 passed · cd slack-bot && npm test → 74 pass

เลือกโมเดลด้วยการวัด ไม่ใช่ด้วยชื่อ

วัดบน corpus จริง โดยกัน thread ไว้ 24 thread ที่โมเดลไม่เคยเห็น *แบบ* = ให้ข้อความหนึ่ง บอกได้ไหมว่าเพื่อนใน thread เดียวกันใกล้กว่าคนแปลกหน้า (50% = เดาสุ่ม)

เทียบ 7 โมเดลบน 1,102 record · held-out 24 thread

โมเดล	แม่นยำ	ชนะได้	COSINE	CONTEXT	สรุป
paraphrase-multilingual-MiniLM	59.4%		0.918	128	ตัวเดิม — อ่อนสุด
paraphrase-multilingual-mpnet	63.1%		0.925	128	แม่นยำ แต่ขยะสูงสุด
multilingual-e5-base	60.6%		0.860	512	กลาง ๆ
multilingual-e5-large	61.4%		0.838	512	ใหญ่ขึ้นเท่าตัว ได้ 0.8 จุด
BAAI/bge-m3	63.7%		0.731	8192	เลือกตัวนี้
เทรนเอง — ข้อมูลสังเคราะห์	61.4%		0.971	128	แพ้
เทรนเอง — ข้อมูลจริง	60.9%		0.964	128	แพ้

เทรนเองสองรอบ และแพ้ทั้งสองรอบ

เทรนจากข้อมูลจริง 936 ข้อความ (2,661 คู่ · loss 5.15 → 2.84) ได้เพิ่มจากฐานแค่ **1.5 จุด** และแพ้ของสำเร็จรูปสามตัว แยกว่านั่นคือมันบีบ cosine ให้พุ่งทั้งกระดาน ขยะได้ 0.964 ทำให้ตัวกรองความเกี่ยวข้องใช้ไม่ได้เลย — เครื่องมือเตือนไว้ตรงตัวก่อนแล้วว่า ถ้าไม่ชนะฐานคือ corpus เล็กเกินไป **ผลนี้คือเครื่องมือทำงานถูก ไม่ใช่พัง**

ตัวกรอง “ไม่มีอะไรตรง” — เปลี่ยนกลไก ไม่ใช่เปลี่ยนเลข

"ใกล้สุดใน corpus เป็น 0.45 โหมด"

1,102 เอกสาร ยังไงก็มีอันที่ตุดล้าย

ทดสอบ 5 โมเดล → กรองขยะได้ 0/5 ทุกตัว

ค่าความใกล้เคียง (BERT) > 0.5 → คะแนนใกล้เคียงกับ 0.5, 0.600, 0.650

และ ความหมายใกล้เคียง > 0.45 → ข้อความที่มีใจความเหมือนกัน

กรองขยะ 3/4 · ไม่เสียคำค้นจริงเลย 0/6

ที่หลุด: อักษรไทยซ้ำ ๆ (ๆ ๆ) ซึ่งมีอยู่จริงในข้อความ

ลองแล้วไม่ผ่าน: z-score ว่า top hit โตจากกลุ่มใหม่ · cross-encoder reranker (cross-encoder ให้คำถามไทยจริง 0.008 ขณะขยะได้ 0.114 — ปฏิเสธด้วยการวัด)

รูป 3 — เกณฑ์ตัวเลขเดียวตอบคำถามนี้ไม่ได้เลย ไม่ใช่เพราะโมเดลไม่ดี การบังคับให้มี "คำที่ตรงกันจริง" ด้วย เป็นตัวที่ทำให้ระบบกล่าตอบว่าไม่มี

04

ใช้งานยังไ

แบ่งตามจังหวะที่คุณอยู่ ไม่ใช่ตามฟีเจอร์ · Python รับจาก `pipeline/` · npm รับจาก `slack-bot/`

ห้าจังหวะที่ใช้จริง

เมื่อไหร่	ทำอะไร	ได้อะไร
ครั้งแรกตอนเริ่ม	<code>python3 -m tam.ingest.users --fetch</code>	ชื่อคนอ่านออก ไม่ใช่ บอ... — แปลตอนแสดงผล ไม่แก้ข้อมูล
เข้าก่อน standup	เปิด <code>/blockers</code> ก่อน แล้ว <code>/</code>	อะไรติด เรียงติดนานสุดก่อน ทุกอันมีข้อความที่พิสูจน์
หลังประชุม	ลาก <code>.vtt</code> จาก Zoom/Meet ใส่ <code>/upload</code>	บทประชุมเข้าไปอยู่ในงานเดียวกับที่แชทคุย
ระหว่างวันใน Slack	<code>/meowtam recall <ข้อความ></code>	หาเรื่องที่จำไม่ได้ว่าใครพูดที่ไหน พร้อมบอกว่าตรงเพราะอะไร
ตอนระบบเข้าใจผิด	คลิกขวาข้อความ → พูกับ ticket	แก้แล้วมันจำ และรอบถัดไปเปลี่ยนตาม — ชนะทุกชั้นการจัดกลุ่ม

พิสูจน์ว่าจริงแล้ว

เขียน override 3 แถว → POST `/api/reindex` → ชื่องานเปลี่ยนจาก `c8d55d2` เป็น `PROJ-901` ทั้งใน dashboard และการ์ดใน Slack แล้วลบกลับได้

ต่างจากต่อ AI เข้า Slack ตรง ๆ ยังไง

เริ่มจากยอมรับก่อนว่าอะไรไม่ใช่จุดต่าง — 936 ข้อความใส่ context ของโมเดลใหญ่ได้สบาย “ข้อมูลเยอะเกิน” ไม่ใช่จุดขาย และเรื่องเข้าใจภาษาไทย โมเดลใหญ่เหนือกว่าตัวที่เราใช้ชัดเจน

1. **กล้าตอบว่า “ไม่มี”** — รายงาน 0 blocked จาก 936 ข้อความ ทั้งที่เป็นฟีดแบ็กเรื่อง การโมเดลว่า “อะไรติด” จะได้คำตอบที่ฟังดูดีทุกครั้ง และไม่มีทางรู้ว่าอันไหนจริง
2. **ทุกคำกล่าวอ้างมี citation ที่ตรวจในโค้ด** — ไม่ใช่ในคำสัญญา อ้างอิงจะถูกตัดและติดป้ายให้เห็น
3. **คนแก่แล้วมันจำ** — และรอด rebuild เพราะชื่องานผูกกับเนื้อหา ไม่ใช่เลขคลาสเตอร์ที่เปลี่ยนทุกครั้ง
4. **ข้อมูลไม่ออกจากเครื่อง** — โมเดลรันในเครื่อง ต้องตั้งค่าเองถึงจะมีอะไรถูกส่งออก
5. **ทำซ้ำได้** — input เดิมให้ผลเดิม ตรวจย้อนได้ว่าทำไมถึงสรุปแบบนั้น

ข้อสรุปเชิงสถาปัตยกรรมคือ **ใช้ทั้งคู่** — ควบกับ retrieval ถือสถานะและหลักฐานเพราะต้อง audit ได้ ส่วนโมเดลภาษาเขียนประโยชน์ เพราะภาษาดีกว่า และ citation ถูกตรวจอยู่แล้ว

ข้อกังวลที่ต้องพูดตรง ๆ

1 · จับ blocker ได้น้อยกว่าที่มีจริงมาก และขอขวดไม่ใช่คำศัพท์

อ่านด้วยมือทั้ง 76 ข้อความที่เข้าข่าย pb blocker จริง **22 ข้อความ** ตอนนั้นระบบรายงานได้แค่ **2 งาน** · หลังแก้ไขอ่าน cue ระดับ **บสรทัด** (ทั้ง ingest/blockers.py และฟัง relation) วันนี้รายงาน **7 จาก 38 งาน** ยังต่ำกว่าที่มีจริง แต่ไม่ใช่ตัวเลขเดิมอีกแล้ว เพราะ topic เป็นคลาสเตอร์แนวเรื่อง กินเวลา median 43 วัน 25 ข้อความ แล้วคำ ack สั้น ๆ คำเดียวตัดสินคลาสเตอร์ 44 ข้อความข้ามหลายสัปดาห์ได้ **งานรอบถัดไปอยู่ที่การจัดกลุ่มและกฎตัดสินสถานะ ไม่ใช่ที่ลิสต์คำ**

2 · ข้อมูลลูกค้าอยู่ในเนื้อหาข้อความ

ในข้อมูลจริงมีอีเมลลูกค้าและเลขตัวถังรถถูกพิมพ์อยู่ในข้อความ โหมดชื่อสมมุติซ่อนแค่ *ชื่อคนส่ง* ไม่ได้ซ่อนสิ่งเหล่านี้ก่อนแคปหน้าจอไปนำเสนอต้องไล่ดูด้วยตาทุกครั้ง

3 · ตัวกรองยังหลุดคำค้นภาษาจริงที่ไม่มีเรื่องนั้นในคลัง

พบตอนสาริต ไม่ใช่ตอนวัด — คำสามัญคำเดียวที่บังเอิญตรงก็ผ่านประตูได้ มีตัวเลือกที่วัดแล้วให้ margin เป็นบวกครั้งแรก แต่ยังไม่ลง เพราะเป็นตัวอย่าง 11 อันที่ผู้พัฒนาเลือกเอง — จะเชื่อได้ต้องมีชุดที่คนอื่นเขียนหรือคำค้นจริงจาก log

4 · ตัวอย่างที่แจกมาสาริตการรวมข้ามที่มาได้ อีกแล้ว

โมเดลใหม่เล็กรวมบทประชุมกับแขกเข้าด้วยกันบนไฟล์ตัวอย่าง — และตรวจแล้วว่า **สิ่งที่โมเดลเก่ารวมนั้นผิด** มันเอาประโยคกักทายภาษาไทย 5 ประโยค (รวมบรรทัดปิดประชุม) มากองกัน โดยระบบเองตั้งชื่อกลุ่มนั้นไม่ได้ ตัวอย่างจึงไม่เคยมีเคสจริงอยู่ในนั้น ต้องเติมเข้าไป — *รอยการตัดสินใจ*

5 · git history ยังมีข้อบกพร่องและ Slack ID จริง

tip สะอาดแล้ว แต่ commit ก่อนหน้ายังมีอยู่ การลบต้อง rewrite history + force push บน public repo ซึ่งทำลายของถาวรและพัง clone ของคนอื่น — *รอยการตัดสินใจ*

6 · ต่อ ticket system แล้ว — ที่เหลือคือ drift ยังพิสูจน์ “ประโยคที่ตัดสินสถานะ” ไม่ได้

YouTrack ถูกอ่านจริงแล้ว (อ่านอย่างเดียว) **197 ใบ · เปิดค้าง 57** และทริกก็อยู่ใน corpus เดียวกับ Slack จึงค้นครั้งเดียวเจอทั้งสองแหล่ง ผลที่ใช้งานได้ทันทีคือ **24 ใบเจียบเกิน 21 วัน** (22 ใบอยู่ *Ready for test*) ซึ่ง Slack ไม่พูดถึงเลย · ส่วน **drift** ที่เทียบสถานะสองฝั่งได้ **3 ใบ** ยังมีข้อจำกัดจริง: ประโยคที่ทำให้ สถานะฝั่ง Slack เป็นอย่างนั้น กับประโยคที่มีคนพิมพ์เลขทริกก็เกิด **ไม่ใช่ข้อความเดียวกันเลย (0 จาก 3)** การ์ดทุกใบจึงติดป้ายบอกและโชว์สองประโยคให้เทียบกันเอง

07

พัฒนาต่อ — เรียงตามผลกระทบที่วัดได้

ลำดับนี้มาจากการวัด ไม่ใช่จากความรู้สึก

ลำดับ ทำอะไร

ทำไมอยู่ตำแหน่งนี้

- ✓ **ล้าง markup ก่อนแมตช์ cue — ทำแล้ว**

ingest/quoted.py ตัด fence, quote, ข้อความขีดฆ่า และ inline code ออกก่อนแมตช์ และ `asserted_lines()` เก็บ**ขอบบรรทัด**ไว้ ทำให้ cue อ่านทีละบรรทัดได้ · ยืนยันว่าให้ผลเหมือนเดิมทุกไบต์บน 1,517 record ก่อนเปลี่ยนกฎ

- ✓ ประตูกันไม่ให้ ack คำเดียวตัดสิน คลัสเตอร์ทั้งก้อน — **ทำแล้ว**

cue ที่อยู่ใต้บรรทัดแรกต้องมีอะไรผูกกับข้อความอีกฝั่ง (anchor ร่วม หรือ เอ่ยถึงคนที่เขียนก่อนหน้า) · ตัด 58 คู่ที่อ่านผิดออกโดยไม่สร้างของใหม่เลย และมีเพียงเจ้าตัวที่ยกเลิกคำประกาศว่าติดของตัวเองได้

- ✓ จัดกลุ่มให้ละเอียดขึ้น — **ทำแล้ว**
(`DEFAULT_RESOLUTION = 4.0`)

วัดสองทางก่อนเปลี่ยน: ที่ 1.0 ก่อนที่แยสุดปน ticket ต่างกัน 19 ใบและงานใหญ่สุด 132 ข้อความ · ที่ 4.0 เหลือ 7 ใบและ 50 ข้อความ แลกกับความครอบคลุม 90% → 85%

- 2 ตัวกรองแบบวัดความครอบคลุมของ คำถาม

ตัวเดียวที่วัดได้ margin เป็นบวก · ต้องมีชุดทดสอบที่คนอื่นเขียนก่อนจะเชื่อ

- ✓ ต่อ YouTrack — **ทำแล้ว** (Notion / Jira ยังไม่ได้ต่อ) อ่านอย่างเดียวนะ 197 ใบเข้า corpus เดียวกัน · `/tracker` · `/api/tracker` · `/meowtam silent` · `/meowtam drift` — ไม่มีอะไรเขียนกลับเข้า YouTrack

- 1 เปิดตารางเวลาอัตโนมัติในช่องจริง “ไม่ต้องถามก็รู้” คือหัวใจ · โค้ดพร้อมและงานเข้าดินบนตัวจับเวลาแล้ว เหลือชื่อ `DIGEST_CHANNEL` ไปช่องจริง — ตั้ง `ENABLE_SCHEDULE=1` หลังชี้ช่องทดสอบผ่านแล้ว

- 3 เทรนโมเดลอีกครั้ง *เมื่อ* ข้อมูลมากพอ อยู่ท้ายสุดเพราะวัดแล้วว่าแพ้ · ถ้าจะกลับมาทำ ต้องได้ข้อมูลหลายเท่า และเปลี่ยนฐานเป็น `bge-m3` ก่อน ไม่ใช่เทรนบนตัวเดิม

ทุกตัวเลขในหน้านี้วัดจากเครื่องที่รันระบบอยู่จริง ณ 19 ส.ค. 2569 23:24 และมีคำสั่งกำกับให้ทำซ้ำได้ · corpus โดทุกเข้าที่งานดึงข้อมูลรัน ตัวเลขจึงเป็นภาพของเวลานั้น · บันทึกการทดลองทั้งหมดรวมทั้งตัวที่แพ้อยู่ใน `docs/EXPERIMENTS.md` · คู่มือใช้งานรายวันอยู่ใน `docs/DAILY_USE.md` · ข้อมูลจริงของบริษัทและชื่อพนักงานไม่ได้ขึ้น git — หน้านี้รายงานเฉพาะตัวเลขรวม